

Tytuł: PingPong

**Autorzy: Mateusz Zybura (MZ),
Gabriel Zawiślak (GZ)**

Ostatnia modyfikacja: 31.08.2026

Spis treści

1. Repozytorium git	2
2. Wstęp	2
3. Specyfikacja	2
3.1. Opis ogólny algorytmu	2
3.2. Tabela zdarzeń	3
4. Architektura	4
4.1. Moduł: top	4
4.1.1. Schemat blokowy	4
4.1.2. Porty	5
a) vga - top_vga, input	5
b) vga – top_vga, output	5
4.1.3. Interfejsy	6
a) paddles	6
b) ball	6
c) score	6
4.2. Rozprowadzenie sygnału zegara	6
5. Implementacja	7
5.1. Lista zignorowanych ostrzeżeń Vivado.	7
5.2. Wykorzystanie zasobów	7
5.3. Marginesy czasowe	8
6. Konfiguracja sprzętu	9
7. Film	9

1. Repozytorium git

Adres repozytorium GITa: https://github.com/gabrielzawislak-code/Ping_Pong_projekt

2. Wstęp

Projekt jest cyfrową implementacją klasycznej gry Ping-Pong (Pong) w trybie multiplayer na dwóch płytkach Basys3. Pomysł wziął się z chęci zrealizowania prostej, ale w pełni interaktywnej gry czasu rzeczywistego, wymagającej jednoczesnej obsługi grafiki VGA, fizyki ruchu (odbicia, przyspieszanie piłki, losowość) oraz komunikacji między dwiema niezależnymi płytkami FPGA.

Każdy z graczy steruje swoją paletką za pomocą przycisków własnej płytki Basys3. Cała logika gry (pozycje obu paletek, pozycja piłki, wynik, przebieg rozgrywki) jest liczona na jednej płytce (MASTER), która jednocześnie wyświetla obraz na monitorze VGA. Druga płytka (player_2) nieprzerwanie odczytuje stan swoich przycisków i przesyła go do MASTERA przez UART, dzięki czemu gracz przy drugiej płytce może sterować swoją paletką w czasie rzeczywistym.

Największym wyzwaniem okazała się komunikacja UART między płytkami. Powracający błąd transmisji zmusił nas do zmodyfikowania koncepcji gry na model, który działa obecnie: Master-Slave. Cała logika rozgrywki — fizyka piłki, pozycje obu paletek i wynik — liczona jest wyłącznie na jednej płytce (MASTER), która jako jedyna wyświetla obraz, natomiast druga płytka (player_2/Slave) pełni rolę czystego urządzenia wejściowego, nieprzerwanie przysyłając do MASTERA stan swoich przycisków. To podejście okazało się prostsze i znacznie stabilniejsze niż pierwotnie zakładana dwustronna synchronizacja pełnego stanu gry między dwiema niezależnie liczącymi płytkami.

Mając świadomość, że uproszczenie architektury do modelu Master-Slave ogranicza funkcjonalnie projekt względem pierwotnego założenia (w pełni niezależne, dwukierunkowo zsynchronizowane płytki, każda z własnym renderowaniem), postanowiliśmy zrekompensować to dodatkową funkcjonalnością możliwą do zrealizowania w pozostałym czasie bez ryzyka dla już działającej komunikacji: sterowanie prędkością piłki za pomocą przełączników sprzętowych. Pięć przełączników na płytce MASTER (SW11–SW15) pozwala w czasie rzeczywistym zwiększyć prędkość piłki odpowiednio o +10%, +20%, +30%, +40% i +50% (efekty się sumują, do maksymalnie +150%), a aktualna wartość boostu jest na bieżąco wyświetlana na ekranie w postaci napisu "speed: +X%" tuż pod górną linią boiska. Ta funkcjonalność nie wymaga żadnej dodatkowej komunikacji między płytkami — działa wyłącznie lokalnie na MASTERZE, niezależnie od modułu fizyki piłki, dzięki czemu mogliśmy ją bezpiecznie dodać mimo napiętego harmonogramu.

3. Specyfikacja

3.1. Opis ogólny algorytmu

1. **Start układu** — reset (asynchroniczne wejście, synchroniczne wyjście z resetu przez dedykowany moduł 'reset_ctrl') ustawia obie płytki w stanie początkowym.

2. **Ekran startowy** — MASTER wyświetla napis "PRESS THE MIDDLE BUTTON TO START". Gra czeka na naciśnięcie środkowego przycisku (BTNC) na MASTERZE.

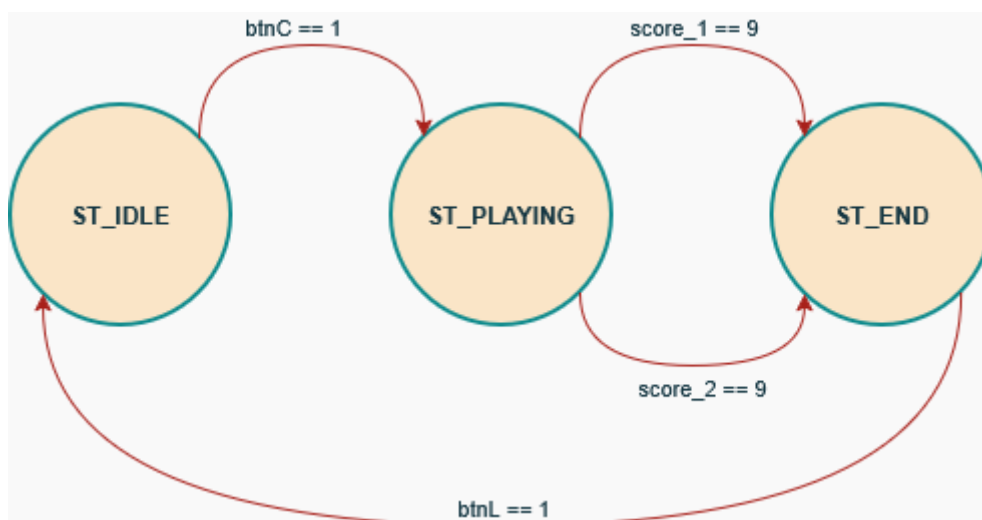
3. **Start rozgrywki** — po naciśnięciu BTNC automat stanu gry (`game\_fsm`) przechodzi w stan `PLAYING`. Piłka oczekuje ok. 1 sekundę na środku ekranu, po czym rusza w losowym kierunku i pod losowym kątem (generowane przez wolnostojący 8-bitowy LFSR, próbkowany w chwili startu — niezależnie od zegara gry, więc wynik jest w praktyce nieprzewidywalny).

4. **Rozgrywka** — obie paletki poruszają się góra/dół (BTNU/BTND): paletka gracza 1 sterowana lokalnymi przyciskami MASTERA, paletka gracza 2 sterowana przyciskami player_2, przesyłanymi ciągle przez UART. Piłka:

- odbija się od widocznej, wsuniętej 12 pikseli od krawędzi ekranu górnej/dolnej granicy boiska,
- odbija się od paletek pod kątem zależnym od miejsca trafienia (trafienie bliżej krawędzi paletki = ostrzejszy kąt),
- przyspiesza o jeden "krok" prędkości po każdym odbiciu od paletki (do limitu), i wraca do wolnego tempa na starcie każdego kolejnego punktu,
- po dotknięciu lewej lub prawej krawędzi ekranu kończy punkt i przyznaje go odpowiedniemu graczowi.

5. **Koniec gry** — pierwszy gracz, który zdobędzie 9 punktów, wygrywa; ekran pokazuje napis "GAMEOVER". Powrót do gry wymaga pełnego resetu (BTNL).

Uwaga: w każdym momencie gry, jest możliwa zmiana prędkości piłki w celu zmiany poziomu trudności, za pomocą pięciu przełączników (SW11-SW15)



3.2. Tabela zdarzeń

Zdarzenie	Kategoria	Reakcja systemu
Naciśnięcie BTNC	Ekran startowy	Start rozgrywki (przejście 'IDLE' -> 'PLAYING')
Naciśnięcie BTNU/BTND (lokalnie lub zdalnie przez "player_2")	Gra	Ruch odpowiedniej paletki w górę lub w dół
Piłka dotyka górnej/dolnej granicy boiska	Gra	Odbicie piłki (zmiana kierunku ruchu pionowego)

Piłka dotyka paletki	Gra	Odbicie piłki pod kątem zależnym od miejsca trafienia, przyspieszenie
Piłka dotyka lewej lub prawej krawędzi ekranu	Gra	Punkt dla przeciwnika, piłka wraca na środek, prędkość resetuje się
Przesunięcie przełącznika (SW11-SW15)	Dowolny stan	Zmiana prędkości piłki (poziom trudności gry)
Wynik jednego z graczy osiąga 9	Gra	Koniec gry, ekran "GAMEOVER"
Naciśnięcie BTNL	Dowolny stan	Pełny reset układu, powrót do ekranu startowego

4. Architektura

Uwaga: dobrze zrobiony projekt zawiera tylko moduły strukturalne (zbudowane z innych modułów) i funkcjonalne (zawierające bloki proceduralne always @). Staramy się nie generować bloków mieszających te dwa typy, o ile to możliwe (głównym kryterium powinna być czytelność kodu).

Uwaga: opisujemy architekturę **tylko głównego modułu oraz rozprowadzenie sygnału zegara**.

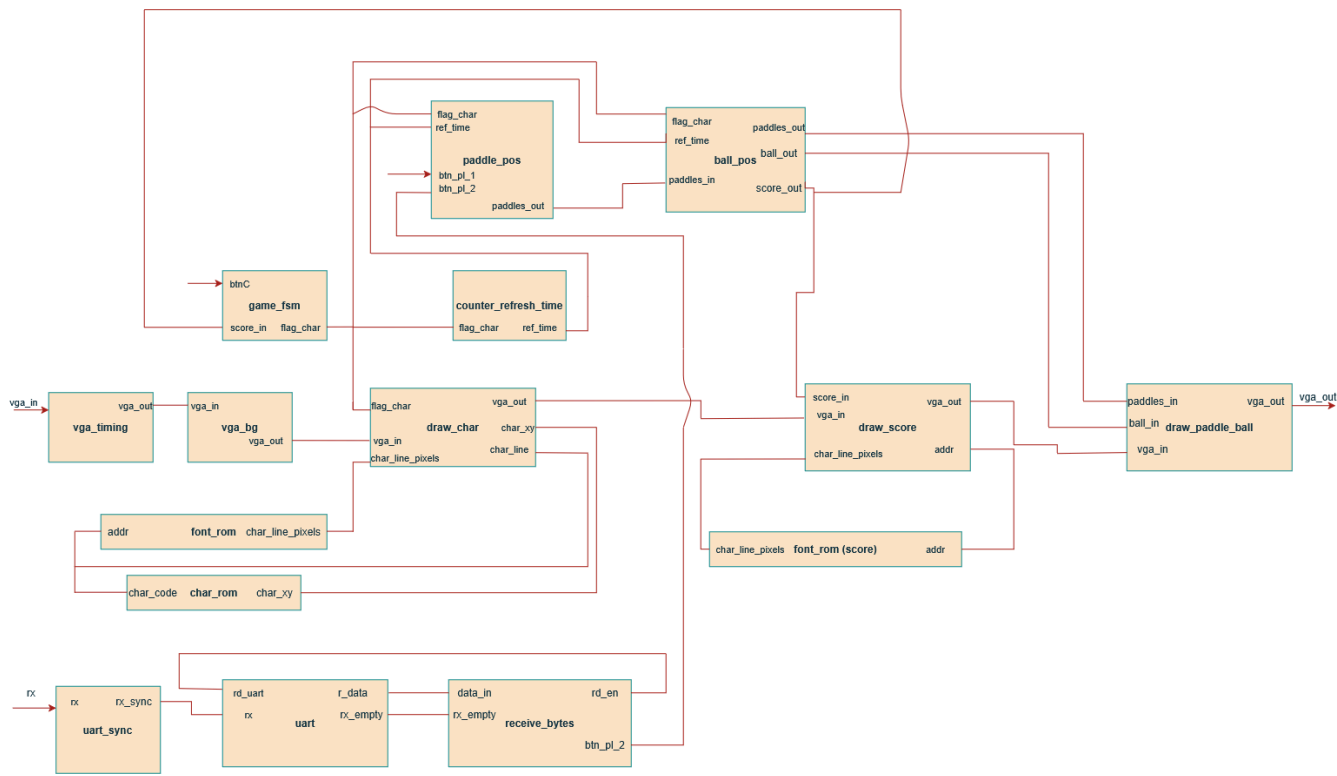
4.1. Moduł: top

Osoba odpowiedzialna: GZ i MZ

4.1.1. Schemat blokowy

Schemat pokazuje moduły składowe top_vga oraz łączące je interfejsy. Gruba czarna linia oznacza interfejs vga_if (grupa sygnałów: hcount, vcount, hsync, vsync, hblnk, vblnk, rgb) przekazywany przez cały potok rysowania. Czerwona pozioma linia to wspólna magistrala flag_char (stan gry z game_fsm), odczytywana przez kilka modułów jednocześnie. Pozostałe cienkie czarne linie to pojedyncze sygnały lub małe grupy sygnałów, podpisane nazwą.

Miejsce na schemat blokowy modułu głównego



4.1.2. Porty

a) *vga - top_vga, input*

nazwa portu	opis
btnC	sygnał z przycisku oznaczonego jako btnC
btn_up	sygnał z przycisku oznaczonego jako btnU
btn_down	sygnał z przycisku oznaczonego jako btnD
rx_pin	sygnał z pinu K17 (Pmod JC)

b) *vga – top_vga, output*

nazwa portu	opis
vs	sygnał synchronizacji pionowej VGA
hs	sygnał synchronizacji poziomej VGA
r	sygnał określający intensywność czerwonej składowej barwy
g	sygnał określający intensywność zielonej składowej barwy
b	sygnał określający intensywność niebieskiej składowej barwy

4.1.3. Interfejsy

a) *paddles*

nazwa sygnału	opis
paddle1_y[10:0]	wertykalna pozycja lewego górnego rogu paletki gracza nr 1
paddle2_y[10:0]	wertykalna pozycja lewego górnego rogu paletki gracza nr 2

b) *ball*

nazwa sygnału	opis
ball_x[10:0]	horyzontalna pozycja lewego górnego rogu piłki
ball_y[10:0]	wertykalna pozycja lewego górnego rogu piłki

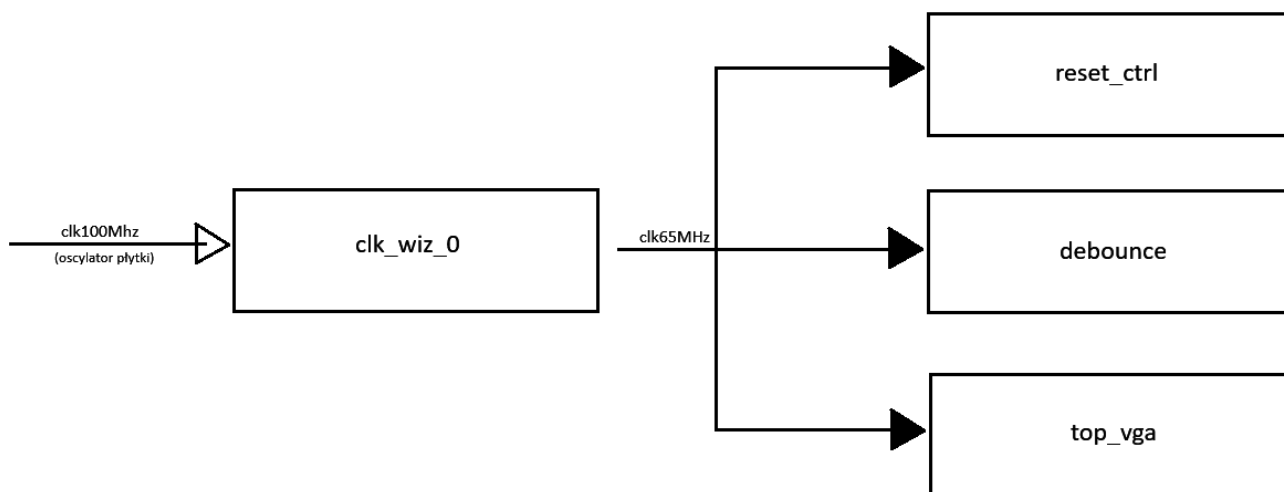
c) *score*

nazwa sygnału	opis
score_1[3:0]	wynik gracza nr 1
score_2[3:0]	wynik gracza nr 2

4.2. Rozprowadzenie sygnału zegara

Osoba odpowiedzialna: MZ i GZ

Jedynym źródłem sygnału zegarowego w projekcie jest wbudowany 100 MHz oscylator płytki Basys3. Moduł generatora zegara (clk_wiz_0, IP Clocking Wizard) jest instancjonowany wyłącznie w module głównym projektu (top_vga_basys3) i generuje jeden sygnał wyjściowy — 65 MHz — używany bez wyjątku we wszystkich pozostałych modułach projektu (logika gry, renderowanie VGA, UART, debounce przycisków, kontroler resetu). Projekt nie wykorzystuje żadnej drugiej częstotliwości zegara — cały układ pracuje w jednej domenie zegarowej, co eliminuje ryzyko problemów z synchronizacją między różnymi zegarami. Ten sam schemat obowiązuje identycznie na obu płytkach (MASTER i player_2), każda z własną instancją clk_wiz_0.



5. Implementacja

5.1. Lista zignorowanych ostrzeżeń Vivado.

Identyfikator ostrzeżenia	Liczba wystąpień	Uzasadnienie
[Board 49-26]	100	Vivado przy tworzeniu projektu próbuje dopasować niepowiązaną płytkę (<code>emb-plus-vpr-4616</code>) z globalnego repozytorium płyt Vivado. Projekt nie ustawia żadnego <code>board_part</code> – tylko konkretny układ (<code>xc7a35tcpg236-1</code>) – to szum środowiska/instalacji Vivado, niezwiązany z żadnym plikiem źródłowym projektu.
[Project 1-5713]	1	Bezpośrednia konsekwencja powyższego – skoro dopasowanie płyty się nie powiodło, właściwość <code>BoardPart</code> zostaje wyzerowana. Nie wpływa to na syntezę pod wskazany part.
[Synth 8-3848]	1	Sygnał <code>vga_time.rgb</code> nie ma sterownika, bo moduł <code>vga_timing</code> (pierwszy etap potoku VGA) generuje tylko sygnały synchronizacji, nie kolor – kolor jest tworzony dopiero w <code>draw_bg</code> , który nigdy nie odczytuje tego pola z wcześniejszego etapu. Sygnał jest martwy, ale bezpiecznie martwy (nigdy nieodczytywany).
[Synth 8-7129]	24	Konsekwencja powyższego – kolejne nieużywane pola współdzielonego interfejsu <code>vga_if</code> , przekazywanego przez cały potok rysowania, nawet gdy dany etap nie potrzebuje wszystkich jego pól. Świadoma decyzja projektowa upraszczająca strukturę potoku kosztem kilku martwych sygnałów.
[Netlist 29-345]	4	Plik IP <code>clk_wiz_0</code> (Clocking Wizard) został wygenerowany dla rodziny UltraScale, a używany jest w projekcie na Artix-7. Vivado automatycznie koryguje to podczas syntezy/implementacji – dotyczy wyłącznie dokładności modelu symulacyjnego prymitywu <code>BUFGCE</code> , nie wpływa na finalny bitstream.
[Synth 8-7257]	1	Na płycie MASTER wejście zapisu modułu <code>uart</code> (<code>wr_uart</code>) jest na stałe podłączone do <code>1'b0</code> , bo MASTER w architekturze Master-Slave nigdy nie nadaje, tylko odbiera. Synteza poprawnie usuwa całkowicie nieużywaną pamięć FIFO nadajnika.
[Synth 8-6014]	1	Bezpośrednia konsekwencja powyższego – rejestr tej samej, nieużywanej pamięci FIFO nadajnika UART.
[Synth 8-3332]	2	Automat stanu nadajnika UART (<code>uart_tx</code>) nigdy nie jest aktywowany z tego samego powodu (MASTER nie nadaje), więc część jego rejestru stanu jest usuwana jako nieużywana.
[Power 33-332]	1	Standardowe ostrzeżenie estymatora mocy o długo utrzymywanym, wysoko rozgałęzionym sygnale resetu – występuje praktycznie w każdym projekcie z globalnym resetem synchronicznym; dotyczy wyłącznie dokładności analizy poboru mocy, nie funkcjonalności układu.

5.2. Wykorzystanie zasobów

Resource	Utilization	Available	Utilization [%]
LUT	756	20800	3.63
LUTRAM	20	9600	0.21
FF	571	41600	1.37
BRAM	1.50	50	3.00
IO	26	106	24.53

BUFG	2	32	6.25
MMCM	1	5	20.00

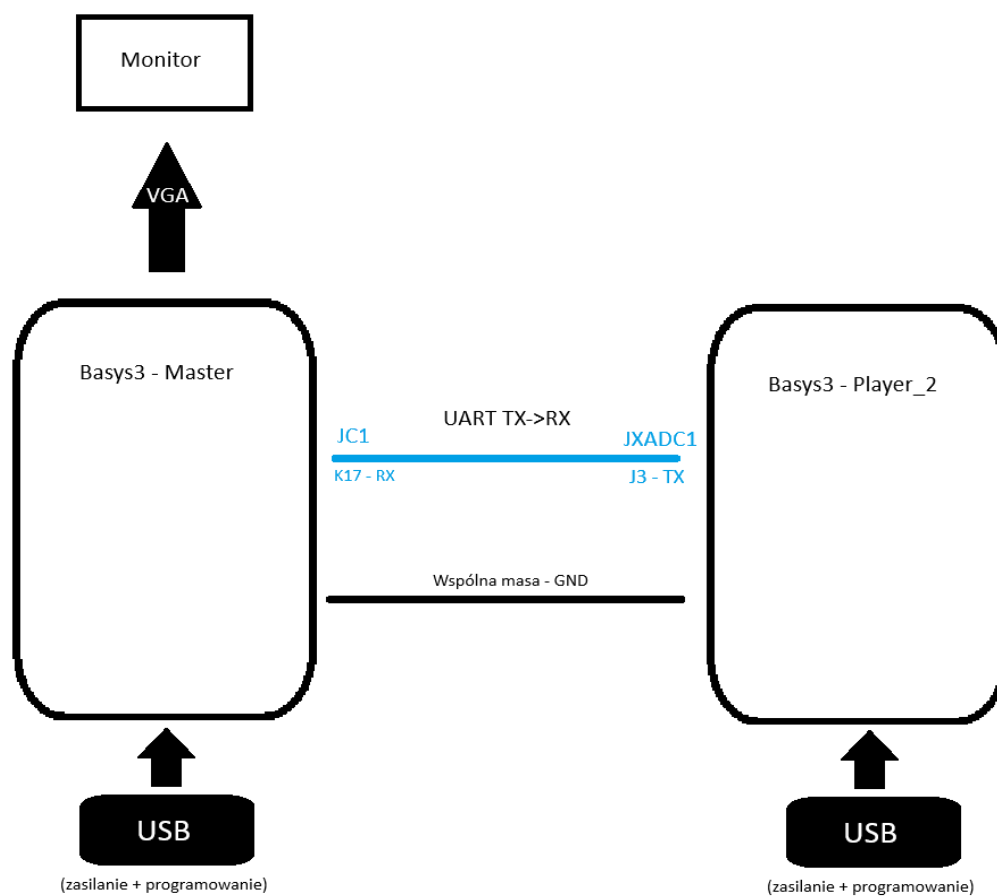
Wykorzystanie zasobów logicznych (LUT, LUTRAM, FF) jest bardzo niskie, co odzwierciedla prostotę logiki cyfrowej projektu. Wyższe wykorzystanie IO (24.53%) wynika z liczby fizycznych sygnałów wejścia/wyjścia (VGA, przyciski, przełączniki, UART), a MMCM (20%, czyli 1 z 5 dostępnych) z pojedynczego wymaganego zegara generowanego przez Clocking Wizard. BRAM (3%) obejmuje kilka pamięci ROM ze znakami tekstowymi (font_rom, char_rom) oraz FIFO modułu UART."

5.3. Marginesy czasowe

	Setup (WNS)	Hold (WHS)
Worst Negative Slack	5,265 ns	0,037 ns
Total Negative Slack	0,000 ns	0,000 ns
Failing Endpoints	0 z 1375	0 z 1375

Analiza czasowa po implementacji (post-route) wykazuje dodatnie marginesy zarówno dla setup (WNS = 5,265 ns), jak i dla hold (WHS = 0,037 ns), przy zerowej liczbie niespełnionych punktów końcowych (0 z 1375 dla obu przypadków). Oznacza to, że projekt spełnia wszystkie zadane ograniczenia czasowe z zapasem, a układ będzie działał poprawnie przy założonej częstotliwości zegara

6. Konfiguracja sprzętu



7. Film.

Link do ściągnięcia filmu:

https://drive.google.com/file/d/1F1F3ql2QsWgdxztQOdl1wUivi-s__ukC/view?usp=sharing